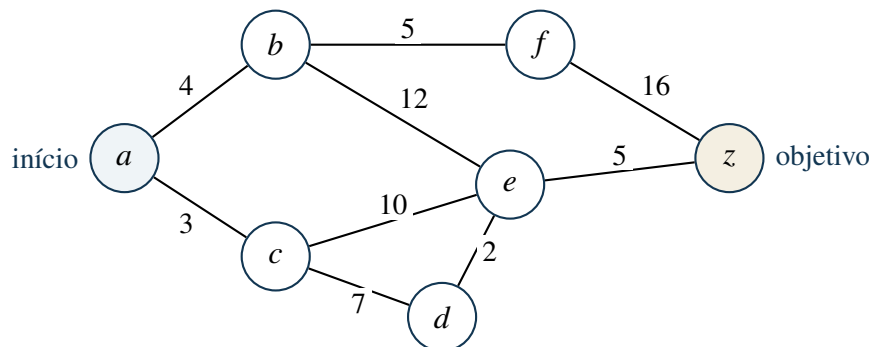


Inteligência Artificial

A* E CONSTRUÇÃO DE HEURÍSTICAS

QUESTÃO 1 — A* PASSO A PASSO

O grafo **não orientado** seguinte é o exemplo dos slides IA-A: o início é a e o objetivo é z . Os números nas ligações são custos, em unidades abstratas. Cada ligação pode ser percorrida nos dois sentidos, com o mesmo custo.



Estado	a	b	c	d	e	f	z
$h(n)$	14	12	11	6	4	11	0

Exercício de consolidação: retoma o exemplo resolvido na apresentação IA-A.

O que se pede

- Calcule $h^*(n)$ para todos os estados e verifique admissibilidade e consistência. Distinga $g(n)$, $h(n)$ e $c(n, m)$.
- Execute A*, mostrando OPEN e CLOSED após cada expansão e após a retirada do objetivo, com a notação $n : f(n)$. Registe também os valores de g , h , f e o predecessor dos estados inseridos ou atualizados.
- Explique as atualizações de e , reconstrua o caminho e conte retiradas e expansões. O estado f é expandido?
- Compare com UCS e procura gulosa (Greedy) neste grafo. Em UCS use as mesmas regras, mas prioridade g ; na procura gulosa use prioridade h e conserve apenas a primeira descoberta. Indique caminhos, custos e ordens de retirada.

Solução proposta

Questão 1 — a) Custos e propriedades da heurística

$g(n)$ é o melhor custo conhecido desde a até n ; $h(n)$ estima o custo ótimo restante até z ; $c(n, m)$ é o custo de uma única ligação. O custo ótimo restante $h^*(n)$ não depende do caminho já percorrido.

Calculando os melhores caminhos até z , obtém-se:

Estado	a	b	c	d	e	f	z
$h(n)$	14	12	11	6	4	11	0
$h^*(n)$	17	17	14	7	5	16	0

Por exemplo, $h^*(c) = 7 + 2 + 5 = 14$ e $h^*(b) = 12 + 5 = 17$. Como $0 \leq h(n) \leq h^*(n)$ para todos os estados, a heurística é **admissível**.

Num grafo não orientado, verificar a consistência em ambos os sentidos de cada ligação equivale a verificar $|h(n) - h(m)| \leq c(n, m)$. Aqui:

Ligação	Verificação	Ligação	Verificação	Ligação	Verificação
$a - b$	$2 \leq 4$	$a - c$	$3 \leq 3$	$b - f$	$1 \leq 5$
$b - e$	$8 \leq 12$	$c - e$	$7 \leq 10$	$c - d$	$5 \leq 7$
$d - e$	$2 \leq 2$	$e - z$	$4 \leq 5$	$f - z$	$11 \leq 16$

Todas as ligações satisfazem a condição e $h(z) = 0$. A heurística é também **consistente**; não serão necessárias reaberturas nesta execução.

b) Evolução de OPEN e CLOSED

Em OPEN, o número depois dos dois-pontos é a prioridade $f(n)$, não apenas $g(n)$ nem apenas $h(n)$. O primeiro elemento é o próximo a retirar.

Operação	OPEN após operação	CLOSED
Inicial	$[a : 14]$	$\emptyset$
Expandir a	$[c : 14, b : 16]$	$\{a\}$
Expandir c	$[b : 16, d : 16, e : 17]$	$\{a, c\}$
Expandir b	$[d : 16, e : 17, f : 20]$	$\{a, c, b\}$
Expandir d	$[e : 16, f : 20]$	$\{a, c, b, d\}$
Expandir e	$[z : 17, f : 20]$	$\{a, c, b, d, e\}$
Retirar z	$[f : 20]$	$\{a, c, b, d, e\}$

Depois de expandir c , b e d têm prioridade 16. FIFO escolhe b , pois este entrou em OPEN antes de d . Ao retirar z , o teste de objetivo termina a procura; z não é expandido nem acrescentado a CLOSED.

Questão 1 — continuação da solução

Registos inseridos ou atualizados

Os valores seguintes mostram a descoberta inicial e as alterações aceites; custos candidatos iguais ou superiores ao melhor custo conhecido são rejeitados.

Momento	Estado	g	h	$f = g + h$	Pai
Inicialização	a	0	14	14	—
Expandir a	b	4	12	16	a
	c	3	11	14	a
Expandir c	d	10	6	16	c
	e	13	4	17	c
Expandir b	f	9	11	20	b
Expandir d	e	12	4	16	d
Expandir e	z	17	0	17	e

c) Melhorias de custo, predecessores e caminho

O estado e é descoberto por c , com $g(e) = 3 + 10 = 13$ e $f(e) = 13 + 4 = 17$. Pela ligação de b para e , o custo candidato é $4 + 12 = 16$, pior do que 13: rejeita-se. Ao expandir d , encontra-se $g(d) + c(d, e) = 10 + 2 = 12 < 13$. Atualizam-se $g(e) = 12$, $pai(e) = d$ e $f(e) = 16$.

Estado	a	b	c	d	e	f	z
g final	0	4	3	10	12	9	17
Predecessor	—	a	a	c	d	b	e

A cadeia de predecessores é $z \leftarrow e \leftarrow d \leftarrow c \leftarrow a$. Portanto:

$$a \rightarrow c \rightarrow d \rightarrow e \rightarrow z, \quad C = 3 + 7 + 2 + 5 = 17.$$

Há **seis retiradas**: a, c, b, d, e, z , e **cinco expansões**: a, c, b, d, e . O estado f é gerado mas permanece em OPEN. O seu nome não deve ser confundido com a função de avaliação $f(n)$.

d) Comparação com UCS e procura gulosa

Para UCS, a evolução da fronteira, agora escrita como $n : g(n)$, é:

Operação	OPEN após operação
Inicial	$[a : 0]$
Expandir a	$[c : 3, b : 4]$
Expandir c	$[b : 4, d : 10, e : 13]$
Expandir b	$[f : 9, d : 10, e : 13]$
Expandir f	$[d : 10, e : 13, z : 25]$
Expandir d	$[e : 12, z : 25]$
Expandir e	$[z : 17]$
Retirar z	$[]$

UCS retira a, c, b, f, d, e, z e encontra o mesmo caminho ótimo de custo 17. Neste caso, terminar ao gerar z devolveria incorretamente custo 25; o custo melhora antes da retirada.

Na procura gulosa, com prioridade h , a sequência de fronteiras é

$$[a : 14] \rightarrow [c : 11, b : 12] \rightarrow [e : 4, d : 6, b : 12] \rightarrow [z : 0, d : 6, b : 12] \rightarrow [d : 6, b : 12].$$

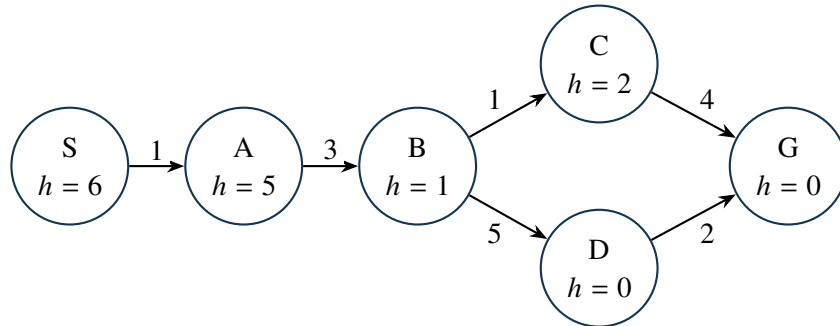
A ordem de retirada é a, c, e, z ; o caminho é $a \rightarrow c \rightarrow e \rightarrow z$, de custo $3 + 10 + 5 = 18$. A procura gulosa expande menos estados neste exemplo, mas devolve uma solução mais cara.

Algoritmo	Prioridade	Custo	Retiradas	Expansões
A*	$g + h$	17	6	5
UCS	g	17	7	6
Gulosa	h	18	4	3

QUESTÃO 2 — CORRIGIR UMA HEURÍSTICA

Adaptação de Berkeley CS188, Summer 2023, *Midterm Review Search*, Q2(c). Grafo, custos e heurísticas preservados; perguntas de verificação acrescentadas. [R1]

O início é S e o objetivo é G. Neste exercício, o grafo é **dirigido**: só são permitidos os sentidos das setas.



Dados fornecidos

Use $h^*(n)$ para o custo ótimo restante de n até G. A admissibilidade exige $h(n) \leq h^*(n)$ para todos os estados. A consistência exige $h(n) \leq c(n, m) + h(m)$ para cada arco $n \rightarrow m$, com $h(G) = 0$.

O que se pede

- Calcule os valores de h^* e determine se a heurística inicial é admissível.
- Verifique a consistência em todos os arcos. Identifique a desigualdade que falha.
- Altere a heurística de **um único estado**, mantendo todas as restantes. Que estado permite tornar a heurística admissível e consistente? Determine o intervalo completo de valores possíveis.
- Explique por que não basta diminuir $h(A)$ para resolver o problema sem alterar mais nenhum valor.

Registro de apoio

Estado	S	A	B	C	D	G
$h(n)$	6	5	1	2	0	0
$h^*(n)$						

Solução proposta

Questão 2 — a) Admissibilidade

Começando no objetivo e calculando para trás:

$$h^*(G) = 0, \quad h^*(C) = 4, \quad h^*(D) = 2,$$

$$h^*(B) = \min(1 + 4, 5 + 2) = 5, \quad h^*(A) = 3 + 5 = 8, \quad h^*(S) = 1 + 8 = 9.$$

Estado	S	A	B	C	D	G
$h(n)$	6	5	1	2	0	0
$h^*(n)$	9	8	5	4	2	0

A heurística inicial é **admissível**: não sobrestima o custo ótimo restante em nenhum estado. Isso não basta, por si só, para garantir consistência.

b) Testar cada arco

Arco	Condição	Resultado
$S \rightarrow A$	$6 \leq 1 + 5$	Verdadeira
$A \rightarrow B$	$5 \leq 3 + 1$	Falsa
$B \rightarrow C$	$1 \leq 1 + 2$	Verdadeira
$B \rightarrow D$	$1 \leq 5 + 0$	Verdadeira
$C \rightarrow G$	$2 \leq 4 + 0$	Verdadeira
$D \rightarrow G$	$0 \leq 2 + 0$	Verdadeira

Logo, a heurística inicial não é consistente.

c) Alterar apenas B

Seja $x = h(B)$. As condições relevantes são:

$$\begin{aligned} 5 &\leq 3 + x \implies x \geq 2 \quad (\text{arco } A \rightarrow B), \\ x &\leq 1 + 2 \implies x \leq 3 \quad (\text{arco } B \rightarrow C), \\ x &\leq 5 + 0 \implies x \leq 5 \quad (\text{arco } B \rightarrow D). \end{aligned}$$

A admissibilidade de B também exige $x \leq h^*(B) = 5$. Intersectando todas as restrições:

Estado B, $2 \leq h(B) \leq 3.$

Todos os outros testes ficam inalterados. Se forem admitidos apenas valores inteiros, as opções são 2 e 3; para valores reais, todo o intervalo fechado é válido.

d) Por que não alterar apenas A?

Mantendo $h(B) = 1$, $A \rightarrow B$ exige $h(A) \leq 4$. Mas $S \rightarrow A$, mantendo $h(S) = 6$, exige $h(A) \geq 5$. As condições são incompatíveis. Alterar S, C, D ou G também não corrige $A \rightarrow B$, pois nenhum desses valores aparece na desigualdade que falha.

QUESTÃO 3 — COMBINAR HEURÍSTICAS

Adaptação de Berkeley CS188, Summer 2023, *Midterm Review Search*, Q3(c)(i). A justificação por desigualdades e o exemplo numérico são desenvolvimentos desta ficha. [R1]

Para o mesmo problema e o mesmo objetivo G , sejam h_1 e h_2 heurísticas admissíveis, não negativas, com $h_1(G) = h_2(G) = 0$. Os custos e as estimativas são valores numéricos na mesma escala de custo.

O que se pede

a) Assinale as combinações que são **sempre** admissíveis. Justifique cada resposta.

Combinação	Sempre admissível	Sem garantia
$h_1 + h_2$	☐	☐
$\frac{1}{2}h_1$	☐	☐
$\frac{1}{2}h_2$	☐	☐
$\frac{1}{2}(h_1 + h_2)$	☐	☐
$h_1 h_2$	☐	☐
$\max(h_1, h_2)$	☐	☐
$\min(h_1, h_2)$	☐	☐

b) Entre as combinações com garantia de admissibilidade, qual fornece um valor pelo menos tão elevado como todas as restantes, em cada estado? Relacione a resposta com a noção de *dominância*.

c) Construa um contraexemplo numérico que mostre por que somar ou multiplicar duas heurísticas admissíveis não preserva, em geral, a admissibilidade.

d) Uma heurística mais informativa garante sempre menor tempo de execução? Que custos devem ser considerados?

Dados fornecidos

Adota-se a convenção $0 \leq h_i(n) \leq h^*(n)$. Dizer que h_A domina h_B significa que $h_A(n) \geq h_B(n)$ para todos os estados, comparando heurísticas admissíveis para o mesmo problema.

Solução proposta

Questão 3 — a) Garantias de admissibilidade

Fixe um estado n e escreva $H = h^*(n)$. Sabemos que $0 \leq h_1(n) \leq H$ e $0 \leq h_2(n) \leq H$.

Combinação	Garantia	Justificação
$h_1 + h_2$	Não	Cada parcela pode aproximar-se de H ; a soma pode ultrapassá-lo.
$\frac{1}{2}h_1$	Sim	$0 \leq \frac{1}{2}h_1 \leq h_1 \leq H$.
$\frac{1}{2}h_2$	Sim	$0 \leq \frac{1}{2}h_2 \leq h_2 \leq H$.
$\frac{1}{2}(h_1 + h_2)$	Sim	$\frac{1}{2}(h_1 + h_2) \leq \frac{1}{2}(H + H) = H$.
$h_1 h_2$	Não	O produto pode ser maior do que H .
$\max(h_1, h_2)$	Sim	Ambos os valores são no máximo H .
$\min(h_1, h_2)$	Sim	O mínimo não excede nenhuma das parcelas.

No objetivo, todas as combinações listadas valem zero. Isso é necessário nesta convenção, mas não elimina as possíveis sobrestimações noutros estados.

b) Máximo e dominância

Para todo o estado, $\max(h_1, h_2)$ é pelo menos tão grande como h_1 , h_2 , as metades, a média e o mínimo. Além disso, continua a não exceder h^* :

$$h_{\max}(n) = \max\{h_1(n), h_2(n)\}.$$

É, portanto, a combinação admissível dominante entre as opções dadas. Esta afirmação compara **informação heurística**, não mede diretamente tempo de execução.

c) Contraexemplo desenvolvido para esta ficha

Considere apenas um arco $S \rightarrow G$ de custo 10. Defina $h_1(S) = 8$, $h_2(S) = 6$ e ambos os valores no objetivo iguais a zero. Como $h^*(S) = 10$, as duas heurísticas são admissíveis. Porém,

$$(h_1 + h_2)(S) = 14 > 10, \quad (h_1 h_2)(S) = 48 > 10.$$

Logo, nenhuma dessas duas operações tem garantia geral. Não se conclui que todas as somas ou todos os produtos sejam inadmissíveis: o contraexemplo refuta apenas a garantia universal.

d) Informação e custo computacional

Uma heurística pode orientar melhor a procura e, ainda assim, ser mais demorada de calcular. O tempo total inclui o cálculo de h , a geração de sucessores, as expansões e a gestão de OPEN e dos restantes registos. Os desempates e a implementação também influenciam as contagens. Por isso, maior informação não garante, sem outras condições, menor tempo em todas as execuções.

QUESTÃO 4 — DISTÂNCIA E CUSTO EM AÇÕES

Adaptação de Berkeley CS188, Spring 2026, *Regular Discussion 1*, Q2(b)–(c). Mantêm-se as regras de movimento e o exemplo do corredor; acrescenta-se uma pergunta sobre o valor zero da heurística. [R2]

Um agente desloca-se numa grelha, orientado para norte, sul, este ou oeste. A velocidade inteira satisfaz $0 \leq v \leq V_{\max}$, com $V_{\max} > 0$. Cada ação custa uma unidade de tempo.

Dados fornecidos

Acelerar: aumentar v em 1 e avançar tantas casas como a nova velocidade. **Desacelerar:** diminuir v em 1 e avançar tantas casas como a nova velocidade. **Virar:** rodar 90° para a esquerda ou direita, apenas com $v = 0$, sem mudar de posição. Não é permitido manter uma velocidade não nula por uma ação de avanço sem alteração de velocidade. Colisões e velocidades fora dos limites são proibidas. O objetivo exige estar na saída **e parado**.

Caso concreto

Considere um corredor horizontal, sem obstáculos, com posições de 0 a 9. O agente começa em $x = 0$, parado e orientado para a direita. A saída está em $x = 9$ e $V_{\max} = 3$.

0	1	2	3	4	5	6	7	8	9
início									saída

O que se pede

- Calcule a distância Manhattan até à saída. Mostre uma sequência legal de seis ações que termine na saída com velocidade zero. Manhattan é sempre admissível quando o custo é o número de ações?
- Construa uma heurística admissível não trivial a partir de $d_M(n, G)$ e de $V_{\max}$. Justifique usando um limite para o deslocamento por ação e calcule-a no estado inicial do corredor.
- A heurística proposta pode valer zero numa posição de saída se o agente ainda estiver em movimento? Isso contradiz a admissibilidade? Que condição deve ser testada para terminar a procura?

Registo de apoio

Após ação	0	1	2	3	4	5	6
Posição x	0						
Velocidade v	0						

Solução proposta

Questão 4 — a) Um contraexemplo à Manhattan em ações

A distância Manhattan inicial é $d_M = |9 - 0| = 9$. Uma sequência legal é acelerar três vezes e desacelerar três vezes:

Passo	Ação	Nova v	Avanço	Posição x
0	Estado inicial	0	—	0
1	Acelerar	1	1	1
2	Acelerar	2	2	3
3	Acelerar	3	3	6
4	Desacelerar	2	2	8
5	Desacelerar	1	1	9
6	Desacelerar	0	0	9

No passo 5, o agente já está na posição da saída, mas ainda não está parado. A última ação é necessária para satisfazer o objetivo.

O plano custa 6 ações; logo, $h^*(n_0) \leq 6 < 9 = d_M(n_0, G)$. Isso basta para mostrar que usar diretamente Manhattan como estimativa do número de ações **não é sempre admissível** neste modelo. Não é necessário provar que o plano de seis ações é ótimo para obter o contraexemplo.

b) Adaptar a estimativa à unidade de custo

Em cada ação, o agente avança no máximo $V_{\max}$ casas e não avança durante uma rotação. Se um plano restante tiver k ações, o seu deslocamento total não pode ultrapassar $kV_{\max}$. Para alcançar a saída é necessário cobrir pelo menos a distância Manhattan. Assim,

$$d_M(n, G) \leq kV_{\max} \implies \boxed{h(n) = \frac{d_M(n, G)}{V_{\max}} \leq k}.$$

A desigualdade aplica-se também ao plano ótimo, pelo que $h(n) \leq h^*(n)$. Ignorar paredes, mudanças de direção e o tempo necessário para acelerar ou parar torna esta estimativa otimista, não uma duração exata.

No início do corredor: $h(n_0) = 9/3 = \boxed{3}$ unidades de tempo. A heurística pode ser fracionária, mesmo que o custo de cada ação seja inteiro.

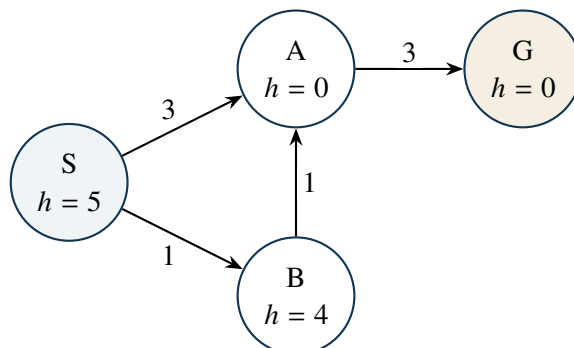
c) Zero não significa necessariamente objetivo

Na posição de saída, $d_M = 0$ e, portanto, $h = 0$, mesmo que $v > 0$. Isso não viola a admissibilidade: zero continua a ser um limite inferior do custo restante. O teste de objetivo tem de verificar o **estado completo**: posição na saída e velocidade zero; não basta testar $h(n) = 0$.

QUESTÃO 5 — REABRIR UM ESTADO DE CLOSED

Exercício de consolidação do exemplo de reabertura apresentado nos slides IA-A.

Considere o grafo **dirigido** seguinte, com início em S e objetivo G. Só são permitidos os sentidos das setas.



Dados fornecidos

Aplique as regras da questão 1: prioridade $f = g + h$, sucessores por ordem alfabética, desempates FIFO e teste do objetivo na retirada de OPEN, antes da expansão. Só aceite melhorias estritas de g . Uma melhoria em OPEN conserva a ordem de inserção; uma reabertura retira o estado de CLOSED e volta a inseri-lo em OPEN, com nova ordem de inserção.

O que se pede

- Calcule h^* e verifique se a heurística é admissível e consistente.
- Execute A* com reabertura. Apresente OPEN e CLOSED após cada expansão e após retirar G. Em OPEN, use a notação $n : (g, h, f)$. Identifique o estado reaberto e as alterações ao seu custo e predecessor.
- Indique o caminho solução, o custo, a ordem de retirada e o número de expansões. Distinga número de expansões de número de estados distintos expandidos.
- Repita a análise para uma versão que ignora qualquer melhoria de um estado já em CLOSED. Que solução devolve? Explique por que a reabertura é necessária neste exemplo.

Solução proposta

Questão 5 — a) Admissível, mas inconsistente

Os custos ótimos restantes são:

$$h^*(G) = 0, \quad h^*(A) = 3, \quad h^*(B) = 1 + 3 = 4, \quad h^*(S) = \min(3 + 3, 1 + 1 + 3) = 5.$$

Em todos os estados, $0 \leq h(n) \leq h^*(n)$, pelo que a heurística é admissível. A consistência verifica-se em $S \rightarrow B$ e $A \rightarrow G$, mas falha em dois arcos:

$$\begin{aligned} S \rightarrow A: \quad 5 &\leq 3 + 0 \quad \text{falsa,} \\ S \rightarrow B: \quad 5 &\leq 1 + 4 \quad \text{verdadeira,} \\ B \rightarrow A: \quad 4 &\leq 1 + 0 \quad \text{falsa,} \\ A \rightarrow G: \quad 0 &\leq 3 + 0 \quad \text{verdadeira.} \end{aligned}$$

Logo, a heurística não é consistente.

b) Execução com reabertura

Em cada entrada $n : (g, h, f)$, o último valor determina a prioridade.

Operação	OPEN após operação	CLOSED
Inicial	$[S : (0, 5, 5)]$	$\emptyset$
Expandir S	$[A : (3, 0, 3), B : (1, 4, 5)]$	$\{S\}$
Expandir A	$[B : (1, 4, 5), G : (6, 0, 6)]$	$\{S, A\}$
Expandir B	$[A : (2, 0, 2), G : (6, 0, 6)]$	$\{S, B\}$
Expandir A novamente	$[G : (5, 0, 5)]$	$\{S, B, A\}$
Retirar G	$[]$	$\{S, B, A\}$

Ao expandir B, encontra-se um caminho melhor para A:

$$g(B) + c(B, A) = 1 + 1 = 2 < 3 = g(A).$$

Atualizam-se $g(A) = 2$ e $\text{pai}(A) = B$. A sai de CLOSED e volta a OPEN com $f(A) = 2$. Na segunda expansão de A, o custo de G melhora de 6 para $2 + 3 = 5$. O predecessor de G continua a ser A.

c) Caminho e contagens

A cadeia final de predecessores é $G \leftarrow A \leftarrow B \leftarrow S$. Assim:

$$\boxed{S \rightarrow B \rightarrow A \rightarrow G}, \quad C = 1 + 1 + 3 = \boxed{5}.$$

A ordem de retirada é S, A, B, A, G : **cinco retiradas**. Há **quatro expansões**, S, A, B, A , mas apenas **três estados distintos expandidos**, S, A e B . G é retirado, mas não expandido nem acrescentado a CLOSED.

d) Execução sem reabertura

Se a melhoria de A for ignorada por este já pertencer a CLOSED, após expandir B mantém-se

OPEN=[$G : (6, 0, 6)$]. Retira-se G e devolve-se:

$$S \rightarrow A \rightarrow G, \quad C = 3 + 3 = \boxed{6}.$$

A ordem de retirada é S, A, B, G . Esta solução é subótima. A admissibilidade, neste exemplo, não impede que A seja expandido primeiro por um caminho mais caro. A reabertura permite propagar a melhoria até ao objetivo.

Referências dos exercícios adaptados

[R1] University of California, Berkeley. CS188, Summer 2023. *Midterm Review Search Solutions*, Q2(c) e Q3(c)(i).

Consultar o enunciado e as soluções de R1 (PDF).

[R2] University of California, Berkeley. CS188, Spring 2026. *Regular Discussion 1 Solutions*, Q2(b)–(c).

Consultar o enunciado e as soluções de R2 (PDF).

As questões 1 e 5 retomam exemplos da apresentação IA-A. As adaptações e os desenvolvimentos das questões 2 a 4 estão identificados nos respectivos enunciados.